

TP2

Image Formation

The objective of this practical work is to study image formation by projecting a 3D point cloud to an image plane using the pinhole camera model.

1 The method

1.1 Pipeline overview

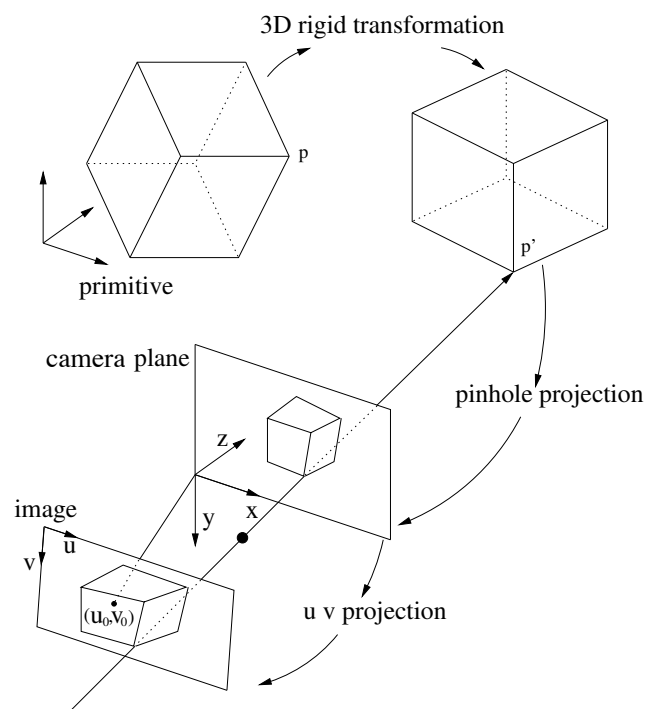


Figure 1: Overview of the pipeline of transformations that an object undergoes to be projected on the camera plane and create an image.

As shown in Figure 1, to form an image, one needs first to place an object (the cube) properly in front of the camera by applying a rigid transformation to it.

Then the object is projected through the pinhole to the camera plane. Finally, the image plane coordinates are converted to uv coordinates (pixel coordinates) and an image is obtained. The 3D rigid transformation, the pinhole projection and the uv projection will be explained below.

1.2 Rigid transform in 3D-space

Transformation of a point in 3D-space:

$$\begin{pmatrix} X_{p'} \\ Y_{p'} \\ Z_{p'} \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R} & t \\ \mathbf{0} & 1 \end{pmatrix} \begin{pmatrix} X_p \\ Y_p \\ Z_p \\ 1 \end{pmatrix} \quad (1)$$

where $\mathbf{R}$ is a 3×3 rotational matrix, t a 3×1 translation vector, and $\mathbf{p}'$ the new point position after the transform.

In practice 3D rotation can be represented in many ways (such as Euler angle, quaternion, and axis-angle) and the calculation of the rotation matrix out of those representations varies. In this TP session we will use yaw, pitch, and roll representation. The function `void computeTrans(...)` to compute the transformation matrix is provided in *imageFormationUtils.h*: it takes the three rotation angles and a translation vector as input and fills the result pointer with the result.

1.3 Pinhole projection

The perspective projection with the origin in the image plane

$$\begin{pmatrix} X_{p'} \\ Y_{p'} \\ 0 \\ 1 + Z_{p'}/f \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1/f & 1 \end{pmatrix} \begin{pmatrix} X_{p'} \\ Y_{p'} \\ Z_{p'} \\ 1 \end{pmatrix} \quad (2)$$

allows to obtain the camera plane coordinates

$$\begin{pmatrix} X_{cam} \\ Y_{cam} \end{pmatrix} = \begin{pmatrix} X_{p'}/(1 + Z_{p'}/f) \\ Y_{p'}/(1 + Z_{p'}/f) \end{pmatrix} \quad (3)$$

1.4 Pixel coordinates u v projection

Now we need to fit a rectangular image to the image plane by choosing the image origin $\begin{pmatrix} u_0 \\ v_0 \end{pmatrix}$ and image resolution α_u and α_v with the unit of meter/pixel so that

we can get pixel coordinates $\begin{pmatrix} u \\ v \end{pmatrix}$ of the projected point:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} X_{cam}/\alpha_u + u_0 \\ Y_{cam}/\alpha_v + v_0 \end{pmatrix}, \quad (4)$$

1.5 3D data file

There are many file formats to store 3D data, such as .ply, .obj, .off, and .stl. What we are going to use is .off. The following is a typical content of an .off file:

```
COFF
6293  9302  0
0.253  3.61 -0.22  243  102  24  255
...
-0.57  2.16  0.24  132  28  55  255
3      0      523  1
...
3      6292  6292  325
```

The first line can be OFF (plain off) or COFF (off with color). The second line specifies the number of vertices, faces and edges in the file. Then comes the point list. In the example above, each point contains xyz coordinates (usually floats), rgb color (usually in the range of 0-255) and transparency value (255 for opaque value, 0 for fully transparent). After the point list, faces and edges can be defined. The first number indicates how many vertices will be used to construct the face or edge (3 for faces, 2 for edges) and the following numbers are the ordered indices in the point list that create the face (or edge). Faces and edges are not used in this TP session. The files .off can be visualized using software such as meshlab.

The examples `frameCube.off` and `human.off` are in the `models` folder.

2 Implementation

Start by copying the implementation of `write_ppm` from your file `Utils/Util.c` from the previous TP (or implement it now). You will need it to save the output image.

Check the file `project_points.c`. It requires multiple arguments regarding the input model, the *camera intrinsic parameters* (f , width, height, u_0 , v_0 , α_u , α_v) and the *camera extrinsic parameters* (gamma, beta, alpha, T_x , T_y , T_z). An option to use a Z-buffer is also required. To see the Usage message test it with:

```
% make
% ./project_points
```

You can call the function¹ with

```
% ./project_points models/cubeFrame.off 1 300 200
150 100 0.1 0.1 0 0 0 0 0 -2 0
```

Assuming the object is already placed in a nice position ($p = p'$), which is done by `centerThePCL` function:

¹Suggestions for an improved starting point are welcome!

1. Implement the pinhole projection to project every point in the object to the camera plane, i.e., implement Eq. 3,
2. Implement the uv projection (Eq. 4) to obtain pixel coordinates.
3. Fill the image coordinates with the point color.

This should create an image.

4. How does the image change if we modify the intrinsic parameters?

Now we want to move the camera, which is done with the re-positionning of the cube:

5. Implement the rigid transformation of the 3D model (Eq. 1) and apply it to the points after `centerThePCL` and before the pinhole projection.
6. How does the image change if we modify the extrinsic parameters?

Now we want to implement the orthogonal projection. Use $f = 0$ in the code².

7. Implement orthogonal projection (you probably need to change the resolution a lot to visualize the result).
8. Now try the 3D points from file `human.off` and implement a method that considers occlusion such that only the front can be seen.
9. Provide a set of intrinsic and extrinsic camera parameters in which the effect of `use_depth` is obvious.
10. **Bonus:** save a .pam file with transparent values in the pixels in which no point is rendered.

²Note that this is a hack. Theoretically $f = 0$ creates a one pixel image and orthogonal camera happens mathematically when $f = \infty$.